

Fundamentos del ciclo de vida del software e ingeniería de requisitos

Breve descripción:

Este componente formativo aborda los fundamentos del ciclo de vida del software y la ingeniería de requisitos, destacando la importancia de identificar, analizar y documentar las necesidades del sistema. Asimismo, presenta los roles involucrados, las técnicas de elicitación y diversas herramientas utilizadas para la captura y modelado de requisitos en proyectos de desarrollo de software.

Septiembre 2026

Tabla de contenido

Introducción	5
1. Fundamentos del ciclo de vida del software	8
1.1. Elementos del ciclo de vida del software	9
1.2. Modelos del ciclo de vida del software	11
1.3. Fases del ciclo de vida del software	14
1.4. Objetivos de las fases del ciclo de vida del software	16
2. Fase de definición de requisitos	18
2.1. Alcance de la fase de requisitos.....	19
2.2. Importancia de la definición de requisitos	20
2.3. Resultados de la fase de requisitos.....	22
3. Requisitos en el desarrollo de software	25
3.1. Requisitos funcionales.....	26
3.2. Requisitos no funcionales.....	28
3.3. Requisitos de restricción	31
4. Ingeniería de requisitos	34
4.1. Áreas de la ingeniería de requisitos.....	36
4.2. Fases de la ingeniería de requisitos	38
5. Elicitación de requisitos	41

5.1.	Planeación de la elicitación	42
5.2.	Fuentes de información	44
5.3.	Técnicas de elicitación	46
5.4.	Instrumentos de recolección de información	49
5.5.	Principios y buenas prácticas.....	51
6.	Roles en la ingeniería de requisitos	54
6.1.	Usuarios y actores	55
6.2.	Stakeholders o partes interesadas	56
6.3.	Cliente líder y dueño del producto	59
6.4.	Equipo de desarrollo y analista.....	61
7.	Herramientas de modelado de requisitos	64
7.1.	Tipos de herramientas de modelado.....	65
7.2.	Características de las herramientas de modelado	68
8.	Herramientas para la captura de requisitos.....	71
8.1.	Diagramas de casos de uso.....	72
8.2.	Historias de usuario	77
8.3.	Storyboard	80
	Síntesis	83
	Glosario	84

Referencias bibliográficas	86
Créditos	87

Introducción

El desarrollo de soluciones tecnológicas exige comprender cómo se estructura y gestiona un proyecto de software desde sus etapas iniciales. En este contexto, el ciclo de vida del software y la ingeniería de requisitos permiten identificar las necesidades de los usuarios y transformarlas en especificaciones claras que orientan el desarrollo de sistemas.

Este componente presenta los fundamentos del ciclo de vida del software, la importancia de la definición de requisitos y los roles que intervienen en este proceso. Asimismo, aborda las técnicas, fuentes de información y herramientas utilizadas para la elicitación, análisis y documentación de requisitos.

A través de estos contenidos, se ofrece una visión general de cómo se identifican, organizan y representan los requisitos en proyectos de software, permitiendo comprender su papel dentro del proceso de desarrollo y la manera en que contribuyen a la construcción de soluciones tecnológicas pertinentes y funcionales.

Para comprender la importancia del contenido y los temas abordados, se recomienda acceder al siguiente video:

Video 1. Fundamentos del ciclo de vida del software e ingeniería de requisitos



[Enlace de reproducción del video](#)

Transcripción del video: Fundamentos del ciclo de vida del software e ingeniería de requisitos

A mediados del siglo XX, cuando comenzaron a desarrollarse los primeros sistemas informáticos complejos, surgió la necesidad de establecer métodos organizados para crear software de forma estructurada y confiable. Investigadores y profesionales del área de la ingeniería de software identificaron que el desarrollo de programas requería procesos definidos que permitieran planificar, analizar y construir sistemas de manera ordenada. De esta necesidad nace el concepto de ciclo de vida del software.

El ciclo de vida del software describe el conjunto de etapas que atraviesa un sistema desde su concepción hasta su implementación y mantenimiento. Entre estas etapas se encuentran la definición de requisitos, el diseño, el desarrollo, las pruebas y

la puesta en funcionamiento. Cada fase tiene objetivos específicos que orientan el trabajo del equipo de desarrollo y permiten asegurar la calidad del producto final.

Dentro de este proceso, la ingeniería de requisitos adquiere un papel fundamental, ya que permite identificar, analizar y documentar las necesidades que el sistema debe satisfacer. Para lograrlo, se emplean diferentes técnicas de elicitación de información, así como herramientas de modelado que facilitan la representación de procesos y funcionalidades.

En la vida real, estos procesos se aplican en múltiples contextos. Por ejemplo, cuando una empresa desea desarrollar una plataforma para gestionar sus ventas, primero se identifican los usuarios del sistema, sus necesidades y las funcionalidades requeridas. Posteriormente, el equipo técnico utiliza herramientas como diagramas de casos de uso, historias de usuario o storyboard para representar cómo funcionará el sistema.

Gracias a estos métodos, el desarrollo de software se convierte en un proceso organizado que permite transformar necesidades reales en soluciones tecnológicas eficientes.

1. Fundamentos del ciclo de vida del software

El desarrollo de software es un proceso estructurado que requiere planificación, organización y control para garantizar que las soluciones tecnológicas respondan adecuadamente a las necesidades de los usuarios y de las organizaciones. En este contexto, el ciclo de vida del software constituye un marco de referencia que orienta las diferentes etapas necesarias para planificar, construir, probar, implementar y mantener un sistema informático.

El ciclo de vida del software describe el conjunto de actividades que se realizan desde la identificación de una necesidad o problema hasta la entrega, operación y mantenimiento del sistema desarrollado. Este enfoque permite organizar el trabajo del equipo de desarrollo, establecer responsabilidades, definir tiempos de ejecución y asegurar la calidad del producto final.

A través del ciclo de vida del software se busca reducir riesgos, mejorar la gestión del proyecto y garantizar que los requerimientos del sistema se comprendan y se implementen correctamente. Además, facilita la comunicación entre los diferentes actores involucrados, como analistas, desarrolladores, usuarios y demás partes interesadas.

Comprender los fundamentos del ciclo de vida del software resulta esencial para cualquier proceso de desarrollo, ya que proporciona una visión integral del proyecto y permite aplicar metodologías y modelos que orientan la creación de soluciones tecnológicas eficientes, sostenibles y alineadas con las necesidades del entorno.

1.1. Elementos del ciclo de vida del software

El ciclo de vida del software está compuesto por un conjunto de elementos que permiten organizar y gestionar el desarrollo de un sistema informático de manera estructurada. Estos elementos orientan las actividades que se realizan durante el proceso de desarrollo, facilitan la coordinación entre los miembros del equipo y contribuyen a garantizar que el producto final cumpla con los requisitos definidos.

Cada elemento cumple una función específica dentro del proceso y permite asegurar que el software sea diseñado, construido, evaluado y mantenido de forma adecuada. En conjunto, estos elementos establecen una guía que facilita la planificación de las actividades, el control del progreso del proyecto y el aseguramiento de la calidad del producto desarrollado.

Entre los principales elementos del ciclo de vida del software se encuentran las actividades, los roles, los artefactos, los procesos y las herramientas, los cuales se describen a continuación.

Tabla 1. Elementos del ciclo de vida del software

Elemento	Descripción
Actividades.	Corresponden a las tareas que se realizan durante el desarrollo del software. Incluyen acciones como analizar requisitos, diseñar el sistema, programar, realizar pruebas, implementar el sistema y efectuar mantenimiento.
Roles.	Representan las responsabilidades que asumen las personas que participan en el proyecto. Algunos roles comunes son analista de

Elemento	Descripción
	requisitos, desarrollador, arquitecto de software, tester, gestor de proyecto y usuario final.
Artefactos.	Son los productos o documentos que se generan durante el proceso de desarrollo. Entre ellos se encuentran los documentos de requisitos, diagramas de diseño, código fuente, planes de prueba, manuales de usuario y reportes técnicos.
Procesos.	Se refieren a la organización estructurada de actividades que se siguen para desarrollar el software. Los procesos establecen cómo se ejecutan las tareas, en qué orden se realizan y cómo se controlan los resultados.
Herramientas.	Son los recursos tecnológicos que apoyan el desarrollo del software. Incluyen entornos de desarrollo, sistemas de control de versiones, herramientas de modelado, plataformas de gestión de proyectos y herramientas de pruebas.

La integración de estos elementos permite que el desarrollo del software se realice de forma organizada y controlada. Cada elemento contribuye a que el proyecto avance de manera estructurada, facilitando la colaboración entre los miembros del equipo y asegurando que el sistema desarrollado cumpla con los estándares de calidad establecidos.

1.2. Modelos del ciclo de vida del software

Los modelos del ciclo de vida del software representan diferentes formas de organizar y gestionar las etapas del desarrollo de un sistema informático. Cada modelo establece una estructura que orienta cómo se planifican, ejecutan y controlan las actividades del proyecto, desde la definición de los requisitos hasta la entrega y mantenimiento del software.

Estos modelos proporcionan una guía metodológica que ayuda a los equipos de desarrollo a gestionar el trabajo de manera más eficiente, definir responsabilidades, controlar los tiempos de ejecución y reducir los riesgos asociados al desarrollo de sistemas. La elección de un modelo depende de factores como el tamaño del proyecto, la complejidad del sistema, el nivel de incertidumbre en los requisitos y la dinámica del equipo de trabajo.

A lo largo del tiempo se han propuesto diversos modelos para organizar el proceso de desarrollo de software. Algunos de los más utilizados se presentan a continuación.

Tabla 2. Principales modelos del ciclo de vida del software

Modelo	Descripción	Características principales
Modelo en cascada.	Es uno de los modelos más tradicionales del desarrollo de software. Las fases del proyecto se ejecutan de manera secuencial, es decir,	Proceso lineal, documentación detallada, adecuado cuando los requisitos están claramente definidos desde el inicio.

Modelo	Descripción	Características principales
	cada etapa comienza cuando la anterior ha finalizado.	
Modelo incremental.	El sistema se desarrolla mediante incrementos o versiones parciales que se van integrando progresivamente hasta completar el producto final.	Permite entregar funcionalidades de manera gradual, facilita la retroalimentación del usuario y reduce riesgos.
Modelo en espiral.	Combina el desarrollo iterativo con el análisis continuo de riesgos. Cada ciclo del proyecto incluye planificación, análisis, desarrollo y evaluación.	Enfoque orientado a la gestión de riesgos, adecuado para proyectos grandes y complejos.
Modelo en V.	Es una extensión del modelo en cascada que relaciona cada fase del desarrollo con una fase correspondiente de pruebas y validación.	Alta trazabilidad entre desarrollo y pruebas, adecuado para proyectos que requieren alta calidad y control.
Modelo ágil.	Se basa en ciclos cortos de desarrollo llamados iteraciones	Flexibilidad, adaptación al cambio, trabajo colaborativo y

Modelo	Descripción	Características principales
	o sprints, donde se entregan pequeñas funcionalidades del sistema de forma continua.	retroalimentación constante del cliente.

Cada uno de estos modelos propone una forma particular de organizar las actividades del desarrollo de software. Aunque comparten etapas similares, difieren en la manera en que se estructuran las fases, la interacción con los usuarios y la forma en que se gestionan los cambios durante el proyecto.

A continuación, se presentan gráficamente algunos modelos del ciclo de vida del software:

- Modelo en cascada.
- Modelo incremental.
- Modelo en espiral.
- Modelo en V.
- Modelo ágil.

Cada modelo ofrece ventajas y limitaciones dependiendo del contexto del proyecto. Por esta razón, las organizaciones seleccionan el modelo de desarrollo que mejor se adapta a sus necesidades, recursos y características del sistema que se desea construir.

En la actualidad, muchos proyectos de software combinan diferentes enfoques o adoptan metodologías ágiles que permiten responder con mayor rapidez a los cambios en los requisitos y a las necesidades del usuario.

1.3. Fases del ciclo de vida del software

El ciclo de vida del software se estructura en un conjunto de fases que orientan el desarrollo de un sistema informático desde la identificación de una necesidad hasta su implementación y mantenimiento. Cada fase agrupa actividades específicas que permiten planificar, diseñar, construir, evaluar y mantener el software de manera organizada.

Estas fases facilitan la gestión del proyecto, ya que permiten dividir el proceso de desarrollo en etapas claramente definidas, lo que contribuye a controlar el avance del proyecto, mejorar la comunicación entre los miembros del equipo y asegurar que el producto final cumpla con los requisitos establecidos.

Aunque el número y la denominación de las fases pueden variar según el modelo de desarrollo utilizado, en términos generales, el ciclo de vida del software comprende las siguientes fases.

Tabla 3. Fases del ciclo de vida del software

Fase	Descripción
Análisis.	En esta fase se identifican y analizan las necesidades del usuario o de la organización. Se recopila información sobre los requisitos que debe cumplir el sistema y se definen los objetivos del proyecto.
Diseño.	Se establece la estructura del sistema, definiendo la arquitectura, los componentes, las bases de datos y la forma en que interactuarán los diferentes elementos del software.
Codificación.	También conocida como fase de desarrollo o programación. En esta etapa, los desarrolladores escriben el código fuente del sistema utilizando lenguajes de programación y herramientas de desarrollo.
Pruebas.	Se realizan diferentes tipos de pruebas para verificar que el sistema funcione correctamente, identificar errores y asegurar que el software cumpla con los requisitos definidos.
Validación.	En esta fase se verifica que el sistema desarrollado satisface las necesidades del usuario y cumple con los objetivos planteados al inicio del proyecto.
Mantenimiento.	Corresponde a las actividades que se realizan después de la implementación del software. Incluye corrección de errores,

Fase	Descripción
	actualización de funcionalidades y mejoras para adaptar el sistema a nuevas necesidades.

Cada una de estas fases cumple un papel fundamental dentro del proceso de desarrollo del software. La adecuada ejecución de estas etapas permite construir sistemas informáticos más confiables, facilitar su mantenimiento y asegurar que respondan de manera efectiva a las necesidades de los usuarios y de las organizaciones.

1.4. Objetivos de las fases del ciclo de vida del software

Las fases del ciclo de vida del software no solo organizan el proceso de desarrollo, sino que también cumplen objetivos específicos que permiten garantizar que el sistema sea construido de manera estructurada, eficiente y alineada con las necesidades del usuario. Cada fase contribuye a alcanzar metas particulares dentro del proyecto, asegurando que el desarrollo del software avance de forma controlada y con resultados verificables.

El establecimiento de objetivos en cada fase permite orientar las actividades del equipo de desarrollo, facilitar la toma de decisiones y reducir los riesgos asociados al proyecto. Asimismo, estos objetivos ayudan a mantener la coherencia entre los requisitos del sistema, las soluciones diseñadas y el producto final que será implementado.

A continuación, se describen los objetivos asociados a las principales fases del ciclo de vida del software:

- a) Análisis:** identificar y comprender las necesidades del usuario o de la organización, definiendo con claridad los requisitos que debe cumplir el sistema.
- b) Diseño:** establecer la arquitectura y la estructura del sistema, definiendo cómo se organizarán sus componentes y cómo interactuarán entre sí.
- c) Codificación:** desarrollar el software mediante la implementación del código fuente que materializa las funcionalidades definidas en el diseño.
- d) Pruebas:** verificar el correcto funcionamiento del sistema, identificando y corrigiendo errores antes de su implementación.
- e) Validación:** confirmar que el sistema desarrollado satisface los requisitos definidos y responde a las necesidades del usuario.
- f) Mantenimiento:** garantizar la continuidad y mejora del sistema mediante la corrección de errores, actualización de funcionalidades y adaptación a nuevos requerimientos.

El cumplimiento de estos objetivos en cada fase permite asegurar un proceso de desarrollo más organizado, mejorar la calidad del software y facilitar su evolución a lo largo del tiempo. De esta manera, el ciclo de vida del software se convierte en una herramienta fundamental para la gestión efectiva de proyectos tecnológicos.

2. Fase de definición de requisitos

La fase de definición de requisitos constituye una de las etapas más importantes dentro del ciclo de vida del software, ya que en ella se identifican, analizan y documentan las necesidades que el sistema debe satisfacer. En esta fase se busca comprender con claridad qué espera el usuario del sistema, cuáles son las funcionalidades que debe ofrecer y qué restricciones o condiciones deben tenerse en cuenta durante su desarrollo.

Durante este proceso se recopila información proveniente de diferentes fuentes, como usuarios, clientes, expertos del dominio y documentos organizacionales. A partir de esta información se establecen los requisitos del sistema, los cuales describen de manera detallada las funciones que el software debe realizar y las características que debe cumplir para responder a las necesidades planteadas.

La correcta definición de los requisitos es fundamental para el éxito de un proyecto de software, ya que permite establecer una base clara para las fases posteriores del desarrollo, como el diseño, la programación y las pruebas. Cuando los requisitos están bien definidos, el equipo de desarrollo puede comprender mejor los objetivos del sistema y reducir la posibilidad de errores, retrabajos o cambios inesperados durante el proceso de construcción del software.

Asimismo, esta fase favorece la comunicación entre los diferentes actores involucrados en el proyecto, como usuarios, analistas, desarrolladores y gestores del proyecto, facilitando la construcción de una visión compartida sobre el sistema que se desea desarrollar. De esta manera, la fase de definición de requisitos se convierte en un elemento clave para garantizar que el software final responda de manera efectiva a las necesidades de la organización y de sus usuarios.

2.1. Alcance de la fase de requisitos

El alcance de la fase de requisitos se refiere al conjunto de actividades orientadas a identificar, comprender, analizar y documentar las necesidades que debe satisfacer el sistema de software. En esta etapa se busca establecer con claridad qué debe hacer el sistema, cuáles son sus funcionalidades principales y qué condiciones o restricciones deben considerarse durante su desarrollo.

Esta fase permite delimitar el problema que se desea resolver mediante el sistema, así como definir los límites del proyecto. De esta manera, se establece qué aspectos serán incluidos dentro del desarrollo del software y cuáles quedarán fuera de su alcance, evitando ambigüedades o interpretaciones incorrectas durante las etapas posteriores del proyecto.

Dentro del alcance de esta fase se desarrollan diversas actividades que permiten comprender las necesidades de los usuarios y transformarlas en requisitos claramente definidos. Entre las principales actividades se encuentran:

- **Identificación de necesidades del usuario**

Consiste en reconocer los problemas, expectativas y objetivos que los usuarios esperan resolver mediante el sistema de software.

- **Recolección de información**

Se obtiene información relevante a partir de diferentes fuentes, como entrevistas con usuarios, análisis de documentos existentes, observación de procesos o reuniones con expertos del dominio.

- **Análisis de requisitos**

Se examina la información recopilada con el fin de comprender las funcionalidades que debe ofrecer el sistema y las restricciones que deben considerarse.

- **Documentación de requisitos**

Los requisitos identificados se organizan y registran de manera estructurada, permitiendo que todos los miembros del equipo comprendan las características que debe tener el sistema.

- **Validación de requisitos**

Se revisan los requisitos junto con los usuarios o las partes interesadas para asegurar que representan correctamente sus necesidades y expectativas.

El alcance de la fase de requisitos permite establecer una base sólida para el desarrollo del software, ya que facilita la comprensión del problema, orienta el trabajo del equipo de desarrollo y contribuye a que el sistema final responda de manera adecuada a las necesidades de los usuarios y de la organización.

2.2. Importancia de la definición de requisitos

La definición de requisitos es una etapa fundamental en el desarrollo de software, ya que permite establecer con claridad las necesidades que el sistema debe satisfacer. Cuando los requisitos se identifican y documentan de manera adecuada, el equipo de desarrollo puede comprender mejor el problema que se desea resolver y diseñar soluciones que respondan a las expectativas de los usuarios.

Una adecuada definición de requisitos contribuye a reducir errores durante el proceso de desarrollo, evitar malentendidos entre los participantes del proyecto y disminuir los costos asociados a cambios o retrabajos en etapas posteriores. Además, facilita la planificación del proyecto, ya que permite estimar con mayor precisión los recursos, el tiempo y el esfuerzo necesarios para desarrollar el sistema.

Asimismo, los requisitos bien definidos sirven como referencia para las fases de diseño, desarrollo y pruebas, ya que establecen los criterios que permitirán verificar si el software cumple con las funcionalidades y características esperadas. De esta manera, la definición de requisitos se convierte en un elemento clave para garantizar la calidad del producto final.

Entre los principales aspectos que evidencian la importancia de la definición de requisitos se encuentran los siguientes:

- **Claridad en los objetivos del sistema:** permite definir qué problema se desea resolver y cuáles son las funcionalidades que debe ofrecer el software.
- **Mejor comunicación entre los participantes del proyecto:** facilita la comprensión compartida entre usuarios, analistas, desarrolladores y demás actores involucrados.
- **Reducción de riesgos en el desarrollo:** disminuye la posibilidad de errores, cambios inesperados o interpretaciones incorrectas durante el proceso de construcción del software.
- **Base para el diseño y desarrollo del sistema:** los requisitos constituyen el punto de partida para definir la arquitectura del sistema y desarrollar sus funcionalidades.

- **Referencia para la validación del software:** permite verificar que el sistema desarrollado cumple con las necesidades y expectativas definidas inicialmente.

Por estas razones, la definición de requisitos es considerada una de las actividades más críticas dentro del ciclo de vida del software, ya que influye directamente en la calidad, funcionalidad y éxito del sistema desarrollado.

2.3. Resultados de la fase de requisitos

Al finalizar la fase de definición de requisitos, se generan diversos productos o entregables que permiten comprender con claridad qué debe hacer el sistema y cuáles son las necesidades que debe satisfacer. Estos resultados sirven como base para el diseño, desarrollo y pruebas del software.

A continuación, se presentan algunos de los principales resultados de esta fase:

Tabla 4. Principales resultados de la fase de requisitos

Resultado	Descripción	Ejemplo
Documento de requisitos del sistema.	Contiene la descripción detallada de las funcionalidades, restricciones y características del sistema.	Documento donde se especifica que el sistema debe permitir registrar usuarios, iniciar sesión y generar reportes.

Resultado	Descripción	Ejemplo
Casos de uso.	Describen cómo interactúan los usuarios con el sistema para realizar determinadas tareas.	Caso de uso “Registrar cliente”, donde el usuario ingresa datos personales y el sistema guarda la información.
Historias de usuario.	Descripciones breves de las necesidades del usuario desde su perspectiva.	“Como administrador, deseo consultar los registros de ventas para analizar el rendimiento del negocio”.
Diagramas de requisitos.	Representaciones gráficas que permiten visualizar las interacciones y funcionalidades del sistema.	Diagrama de casos de uso que presenta la relación entre el usuario y las funciones del sistema.
Validación de requisitos.	Confirmación de que los requisitos definidos reflejan las necesidades de los interesados.	Revisión del documento de requisitos por parte del cliente y aprobación para iniciar el diseño del sistema.

En conjunto, estos resultados permiten establecer una base clara y estructurada para las siguientes etapas del desarrollo del software. La correcta documentación y validación de los requisitos facilita la comprensión del sistema que se desea construir, reduce ambigüedades y contribuye a que el equipo de desarrollo cuente con la información necesaria para avanzar hacia las fases de diseño, construcción y pruebas de manera organizada y coherente.

3. Requisitos en el desarrollo de software

Los requisitos en el desarrollo de software representan las necesidades, condiciones y características que un sistema debe cumplir para satisfacer las expectativas de los usuarios y los objetivos de una organización. Estos requisitos describen tanto las funciones que el sistema debe realizar como las condiciones bajo las cuales debe operar.

La correcta identificación y documentación de los requisitos constituye una de las actividades más importantes dentro del proceso de desarrollo de software, ya que establece las bases para las fases de diseño, implementación, pruebas y mantenimiento del sistema. Cuando los requisitos se definen de manera clara y estructurada, se facilita la comunicación entre los diferentes actores del proyecto y se reduce el riesgo de errores durante el desarrollo.

En general, los requisitos se clasifican en diferentes tipos según el aspecto del sistema que describen. Entre los más comunes se encuentran los requisitos funcionales, que definen las funciones o servicios que el sistema debe ofrecer; los requisitos no funcionales, que establecen las condiciones de calidad o restricciones de operación del sistema; y los requisitos de restricción, que imponen limitaciones técnicas, normativas o tecnológicas que deben cumplirse durante el desarrollo.

Esta clasificación permite organizar la información de manera estructurada y facilita el análisis, la documentación y la validación de los requisitos durante el proceso de desarrollo del software.

3.1. Requisitos funcionales

Los requisitos funcionales describen las funciones, servicios y comportamientos que el sistema debe proporcionar para satisfacer las necesidades de los usuarios y cumplir con los objetivos del negocio. En otras palabras, definen qué debe hacer el sistema frente a determinadas entradas o acciones realizadas por los usuarios.

Estos requisitos especifican las operaciones que el sistema debe ejecutar, la forma en que procesa la información y las respuestas que debe generar ante diferentes situaciones. Por esta razón, constituyen uno de los elementos centrales en la definición del sistema, ya que permiten comprender las funcionalidades que serán implementadas durante el desarrollo del software.

Generalmente, los requisitos funcionales se redactan de forma clara y precisa utilizando expresiones como “el sistema debe permitir”, “el sistema debe registrar” o “el sistema debe generar”. Este tipo de redacción facilita su comprensión y permite que posteriormente se definan los casos de prueba que verificarán si las funcionalidades han sido implementadas correctamente.

A continuación, se presentan algunas categorías comunes de requisitos funcionales que suelen identificarse durante el desarrollo de un sistema de software:

Tabla 5. Categorías de requisitos funcionales

Categoría	Descripción	Ejemplo
Autenticación y control de acceso.	Funcionalidades relacionadas con la identificación de usuarios y	El sistema debe permitir que los usuarios inicien sesión mediante usuario y

Categoría	Descripción	Ejemplo
	la gestión de permisos dentro del sistema.	contraseña y bloquear la cuenta después de tres intentos fallidos.
Gestión de datos (CRUD).	Funciones que permiten crear, consultar, actualizar y eliminar información dentro del sistema.	El sistema debe permitir registrar, consultar, modificar y eliminar información de clientes.
Procesamiento y cálculo.	Operaciones automáticas que transforman o procesan datos según reglas definidas por el negocio.	El sistema debe calcular automáticamente el valor total de una compra, incluyendo impuestos y descuentos.
Reportes y consultas.	Funcionalidades que permiten generar informes o consultar información almacenada en el sistema.	El sistema debe generar reportes de ventas por rango de fechas en formato PDF o Excel.
Notificaciones y alertas.	Mecanismos mediante los cuales el sistema comunica eventos o cambios importantes a los usuarios.	El sistema debe enviar una notificación por correo electrónico cuando se

Categoría	Descripción	Ejemplo
		registre una nueva solicitud.

La correcta definición de los requisitos funcionales permite que el equipo de desarrollo comprenda con claridad las funcionalidades que el sistema debe implementar. Además, facilita la planificación del desarrollo, la elaboración de casos de prueba y la validación del sistema, garantizando que el software responda adecuadamente a las necesidades de los usuarios y de la organización.

3.2. Requisitos no funcionales

Los requisitos no funcionales describen las condiciones, características y restricciones que determinan cómo debe funcionar el sistema. A diferencia de los requisitos funcionales, que especifican las acciones o servicios que el sistema debe realizar, los requisitos no funcionales establecen criterios relacionados con el rendimiento, la seguridad, la disponibilidad, la usabilidad y otros aspectos de calidad del software.

Este tipo de requisitos define el comportamiento global del sistema y establece parámetros que permiten evaluar su desempeño y calidad durante su operación. En muchos casos, los requisitos no funcionales determinan el nivel de eficiencia, confiabilidad y seguridad que debe tener el sistema para cumplir con las expectativas de los usuarios y de la organización.

Para que puedan ser evaluados correctamente, los requisitos no funcionales deben formularse de manera medible y verificable, especificando valores concretos o condiciones que permitan comprobar su cumplimiento durante las pruebas del sistema.

A continuación, se presentan algunas de las categorías más comunes de requisitos no funcionales:

Tabla 6. Categorías de requisitos no funcionales

Categoría	Descripción	Ejemplo
Rendimiento.	Se refiere a la velocidad de respuesta del sistema, su capacidad de procesamiento y el tiempo necesario para ejecutar determinadas operaciones.	El sistema debe responder las consultas de los usuarios en menos de 2 segundos.
Disponibilidad.	Indica el tiempo durante el cual el sistema debe estar operativo y accesible para los usuarios.	El sistema debe estar disponible el 99.9 % del tiempo durante el año.
Seguridad.	Establece mecanismos de protección de la información y control de acceso al sistema.	Las contraseñas deben almacenarse mediante mecanismos de cifrado seguros.

Categoría	Descripción	Ejemplo
Usabilidad.	Hace referencia a la facilidad con la que los usuarios pueden aprender a utilizar el sistema e interactuar con él.	El sistema debe permitir que los usuarios completen el registro en menos de tres minutos.
Escalabilidad.	Capacidad del sistema para soportar un aumento en la cantidad de usuarios o en el volumen de datos sin afectar su rendimiento.	El sistema debe soportar hasta 10.000 usuarios concurrentes.
Portabilidad.	Capacidad del sistema para funcionar en diferentes plataformas, dispositivos o sistemas operativos.	La aplicación debe funcionar en dispositivos móviles, tabletas y navegadores web modernos.
Mantenibilidad.	Facilidad para realizar modificaciones, correcciones o mejoras en el software después de su implementación.	El sistema debe permitir actualizaciones del software sin afectar la operación del servicio.
Accesibilidad.	Permite que el sistema pueda ser utilizado por personas con	La interfaz debe cumplir con estándares de accesibilidad para personas con discapacidad visual.

Categoría	Descripción	Ejemplo
	diferentes capacidades o limitaciones.	

La identificación adecuada de los requisitos no funcionales permite garantizar que el sistema no solo cumpla con las funciones solicitadas, sino que también ofrezca un nivel adecuado de calidad, seguridad y eficiencia. Estos requisitos influyen directamente en las decisiones de diseño, arquitectura y tecnología utilizadas durante el desarrollo del software.

3.3. Requisitos de restricción

Los requisitos de restricción establecen limitaciones o condiciones que deben cumplirse durante el desarrollo o funcionamiento del sistema de software. Estas restricciones pueden estar relacionadas con aspectos técnicos, normativos, organizacionales o de recursos, y determinan el marco dentro del cual debe diseñarse e implementarse el sistema.

A diferencia de los requisitos funcionales y no funcionales, que describen lo que el sistema debe hacer y cómo debe comportarse, los requisitos de restricción definen condiciones externas o internas que limitan las decisiones de diseño, las tecnologías a utilizar o las formas de implementación del sistema.

Este tipo de requisitos es importante porque permite asegurar que el software se desarrolle respetando normas legales, políticas organizacionales, estándares

tecnológicos o limitaciones de infraestructura. Su adecuada identificación evita problemas futuros relacionados con el incumplimiento de regulaciones, incompatibilidades tecnológicas o dificultades de integración con otros sistemas.

Las restricciones pueden presentarse en diferentes ámbitos, como se describe a continuación:

Tabla 7. Tipos de requisitos de restricción

Tipo de restricción	Descripción	Ejemplo
Tecnológica.	Limita las tecnologías, plataformas o herramientas que pueden utilizarse en el desarrollo del sistema.	El sistema debe desarrollarse utilizando tecnología de código abierto y ejecutarse en servidores Linux.
Legal o normativa.	Establece el cumplimiento de leyes, reglamentos o normas vigentes relacionadas con el manejo de información o el funcionamiento del sistema.	El sistema debe cumplir con la Ley 1581 de 2012 sobre protección de datos personales en Colombia.
Organizacional.	Define políticas internas de la organización que deben respetarse durante el	El sistema debe integrarse con los sistemas corporativos existentes de la organización.

Tipo de restricción	Descripción	Ejemplo
	desarrollo o implementación del sistema.	
Operativa.	Determina condiciones relacionadas con la infraestructura o el entorno donde operará el sistema.	El sistema debe funcionar en entornos con conectividad limitada a Internet.
Económica.	Impone límites relacionados con el presupuesto disponible para el desarrollo o mantenimiento del sistema.	El sistema debe desarrollarse utilizando herramientas que no requieran licencias comerciales adicionales.

La definición clara de los requisitos de restricción permite orientar el proceso de desarrollo dentro de un marco de condiciones previamente establecidas. Estas restricciones influyen en la planificación del proyecto, en la selección de tecnologías y en las decisiones de diseño, garantizando que el sistema desarrollado sea viable desde el punto de vista técnico, organizacional y legal.

4. Ingeniería de requisitos

La ingeniería de requisitos es una disciplina fundamental dentro del desarrollo de software que se encarga de identificar, analizar, documentar, validar y gestionar los requisitos de un sistema. Su objetivo principal consiste en comprender con claridad las necesidades de los usuarios y traducirlas en especificaciones que orienten el diseño y la construcción del software.

Esta disciplina actúa como un puente entre los usuarios, los clientes y el equipo de desarrollo, permitiendo que las expectativas del negocio se transformen en soluciones tecnológicas viables. Cuando los requisitos se definen de forma clara y estructurada, el equipo de desarrollo puede comprender mejor el problema que debe resolverse, lo que facilita la toma de decisiones durante el diseño y la implementación del sistema.

Uno de los principales retos en el desarrollo de software consiste en interpretar correctamente las necesidades de los usuarios. En muchos proyectos, los clientes expresan sus requerimientos de forma general o ambigua, lo que puede generar interpretaciones incorrectas si no se aplica un proceso sistemático de análisis. La ingeniería de requisitos permite reducir estos riesgos mediante el uso de métodos, técnicas y herramientas que facilitan la comunicación entre todas las partes involucradas.

Dentro de esta disciplina se realizan diversas actividades orientadas a comprender el problema, definir las funcionalidades del sistema y establecer las condiciones bajo las cuales debe operar. Estas actividades permiten producir documentos, modelos y especificaciones que sirven como base para las siguientes etapas del ciclo de vida del software.

Entre los principales objetivos de la ingeniería de requisitos se encuentran los siguientes:

- a) Comprender las necesidades y expectativas de los usuarios y de la organización.
- b) Definir de manera clara y estructurada las funcionalidades que debe ofrecer el sistema.
- c) Establecer criterios que permitan verificar y validar el funcionamiento del software.
- d) Facilitar la comunicación entre los diferentes actores involucrados en el proyecto.
- e) Reducir riesgos asociados a interpretaciones incorrectas o cambios tardíos en los requisitos.

La ingeniería de requisitos también contribuye a mejorar la calidad del software, ya que permite detectar inconsistencias, omisiones o ambigüedades en las especificaciones antes de iniciar las etapas de diseño y desarrollo. De esta manera, se disminuyen los costos asociados a correcciones posteriores y se incrementa la probabilidad de que el sistema final cumpla con los objetivos del proyecto.

En el contexto del desarrollo moderno de software, la ingeniería de requisitos se aplica tanto en metodologías tradicionales como en enfoques ágiles, adaptando sus técnicas y prácticas a las características del proyecto. En metodologías ágiles, por ejemplo, los requisitos suelen documentarse mediante historias de usuario, mientras que en enfoques tradicionales se utilizan especificaciones más detalladas.

4.1. Áreas de la ingeniería de requisitos

La ingeniería de requisitos se organiza en diferentes áreas que agrupan las actividades necesarias para identificar, analizar, documentar, validar y gestionar los requisitos de un sistema de software. Cada una de estas áreas cumple una función específica dentro del proceso y permite estructurar el trabajo del equipo de desarrollo de manera ordenada.

Estas áreas facilitan la comprensión de las necesidades del usuario, la definición precisa de los requisitos y el control de los cambios que puedan presentarse durante el desarrollo del proyecto. En conjunto, permiten garantizar que el software responda adecuadamente a los objetivos del negocio y a las expectativas de los usuarios.

Las principales áreas de la ingeniería de requisitos se presentan a continuación:

Tabla 8. Áreas de la ingeniería de requisitos

Área	Descripción	Ejemplo
Elicitación de requisitos.	Consiste en la identificación y recopilación de las necesidades, expectativas y problemas que los usuarios esperan resolver mediante el sistema.	Realizar entrevistas con los usuarios para identificar las funcionalidades que debe incluir el sistema.
Análisis de requisitos.	Implica examinar la información obtenida para comprender los requisitos, detectar inconsistencias, eliminar	Analizar los requisitos recopilados para determinar cuáles son críticos y cuáles

Área	Descripción	Ejemplo
	ambigüedades y priorizar las funcionalidades del sistema.	pueden implementarse en fases posteriores.
Especificación de requisitos.	Se encarga de documentar los requisitos de manera clara, estructurada y comprensible para todos los miembros del proyecto.	Elaborar un documento de especificación de requisitos del software donde se describen las funcionalidades del sistema.
Validación de requisitos.	Busca confirmar que los requisitos definidos reflejan realmente las necesidades del usuario y que son correctos, completos y comprensibles.	Revisar los requisitos con los usuarios para verificar que representan adecuadamente sus necesidades.
Gestión de requisitos.	Permite controlar los cambios que puedan surgir en los requisitos durante el desarrollo del proyecto, asegurando su trazabilidad y actualización.	Registrar cambios solicitados por el cliente y evaluar su impacto en el proyecto.

Estas áreas se encuentran estrechamente relacionadas y se desarrollan de forma iterativa durante el proyecto. La información obtenida en una etapa puede requerir

revisiones o ajustes en otras áreas, lo que permite mejorar progresivamente la definición de los requisitos.

Una adecuada aplicación de estas áreas contribuye a reducir errores en la interpretación de los requisitos, mejorar la comunicación entre los actores del proyecto y facilitar la construcción de sistemas de software que respondan de manera efectiva a las necesidades del usuario.

4.2. Fases de la ingeniería de requisitos

La ingeniería de requisitos se desarrolla mediante una serie de fases que permiten identificar, analizar, documentar y controlar los requisitos de un sistema de software. Estas fases organizan el trabajo del equipo de desarrollo y facilitan la comprensión de las necesidades de los usuarios, garantizando que el sistema cumpla con los objetivos del proyecto.

Aunque en la práctica algunas actividades pueden realizarse de manera iterativa, las fases de la ingeniería de requisitos suelen seguir una secuencia lógica que orienta el proceso de definición de los requisitos.

A continuación, se describen las principales fases:

a) Elicitación de requisitos

Corresponde al proceso de recopilación de información sobre las necesidades, expectativas y problemas que el sistema debe resolver. En esta fase se utilizan diferentes técnicas como entrevistas, encuestas, observación del trabajo de los usuarios o análisis de documentos existentes.

b) Análisis de requisitos

Consiste en examinar y organizar la información obtenida durante la elicitación. El objetivo es comprender los requisitos, identificar inconsistencias, resolver ambigüedades y establecer prioridades entre las diferentes funcionalidades del sistema.

c) Especificación de requisitos

En esta fase, los requisitos se documentan de forma clara, estructurada y comprensible para todos los miembros del proyecto. La especificación puede incluir descripciones textuales, modelos, diagramas o historias de usuario que permitan representar el comportamiento esperado del sistema.

d) Validación de requisitos

Tiene como propósito verificar que los requisitos definidos representan correctamente las necesidades del usuario y que son completos, coherentes y viables desde el punto de vista técnico. Esta fase suele realizarse mediante revisiones, prototipos o reuniones de validación con los stakeholders.

e) Gestión de requisitos

Se encarga de controlar y administrar los cambios que puedan surgir en los requisitos durante el desarrollo del proyecto. Incluye actividades como el seguimiento de modificaciones, el análisis de impacto y la actualización de la documentación correspondiente.

En conjunto, estas fases permiten estructurar el proceso de definición de requisitos, mejorar la comunicación entre los actores del proyecto y asegurar que el software desarrollado responda de manera efectiva a las necesidades del usuario y de la organización.

Uno de los ejemplos más conocidos sobre la importancia de una correcta definición de requisitos ocurrió durante el desarrollo del sistema de equipaje del aeropuerto internacional de Denver en la década de 1990. El proyecto buscaba automatizar completamente el manejo de equipajes mediante un complejo sistema informático y mecánico. Sin embargo, debido a problemas en la definición y gestión de los requisitos, el sistema presentó múltiples fallos, retrasos y sobrecostos.

El proyecto se retrasó aproximadamente 16 meses y generó pérdidas superiores a 560 millones de dólares, convirtiéndose en un caso ampliamente estudiado en ingeniería de software. Este ejemplo evidencia que una inadecuada gestión de requisitos puede afectar seriamente el éxito de un proyecto tecnológico.

Por esta razón, la ingeniería de requisitos se considera una de las etapas más críticas dentro del desarrollo de software, ya que permite comprender con claridad qué necesita el usuario y cómo debe responder el sistema a dichas necesidades.

5. Elicitación de requisitos

La elicitación de requisitos corresponde al proceso mediante el cual se identifican, recopilan y comprenden las necesidades, expectativas y restricciones de los interesados en relación con un sistema o producto de software. Esta etapa permite descubrir qué debe hacer el sistema, cómo debe comportarse y cuáles son las condiciones bajo las cuales debe operar.

En el contexto de la ingeniería de software, la elicitación se considera una de las actividades más importantes dentro de la ingeniería de requisitos, ya que de la calidad de la información obtenida dependen las fases posteriores de análisis, especificación y validación.

Durante este proceso se interactúa con diferentes actores, como usuarios finales, clientes, expertos del dominio, desarrolladores y responsables del negocio. Cada uno aporta información valiosa sobre el funcionamiento esperado del sistema, los problemas actuales y las oportunidades de mejora.

La elicitación no consiste únicamente en preguntar qué necesita el cliente, sino también en analizar el contexto organizacional, identificar requisitos implícitos, resolver ambigüedades y descubrir necesidades que los propios usuarios pueden no haber expresado de forma explícita. Para lograrlo, se utilizan diversas técnicas de investigación, comunicación y análisis que facilitan la comprensión integral del problema.

Además, este proceso suele ser iterativo, lo que significa que la información recopilada se revisa, valida y complementa continuamente a medida que se obtiene una mejor comprensión del sistema y de su entorno. De esta manera, la elicitación

contribuye a reducir errores tempranos, mejorar la calidad de los requisitos y aumentar la probabilidad de éxito del proyecto de software.

5.1. Planeación de la elicitación

La planeación de la elicitación corresponde al proceso mediante el cual se organiza y prepara la obtención de información necesaria para identificar y comprender los requisitos de un sistema. En esta etapa se establecen los objetivos de la recolección de información, se identifican los actores involucrados y se determinan las técnicas y recursos que se utilizarán para obtener los requisitos de manera estructurada y eficiente.

Una adecuada planeación permite optimizar el tiempo, reducir ambigüedades y garantizar que la información recopilada sea pertinente para el desarrollo del software. Además, facilita la coordinación entre analistas, usuarios, expertos del dominio y demás interesados, lo que contribuye a construir una visión compartida del sistema que se desea desarrollar.

Dentro de la planeación de la elicitación se consideran varios aspectos fundamentales:

a) Definición de objetivos de la elicitación

Se determina qué información se necesita obtener y cuál es el alcance del sistema o producto a desarrollar.

b) Identificación de los interesados (stakeholders)

Se reconocen las personas o grupos que interactúan con el sistema o que tienen interés en su desarrollo, como usuarios finales, administradores, clientes o expertos del dominio.

c) Selección de técnicas de elicitación

Se definen los métodos que se utilizarán para recolectar información, como entrevistas, talleres, cuestionarios o análisis de documentos.

d) Planificación de actividades

Se establece un cronograma de reuniones, sesiones de trabajo y actividades de recolección de información.

e) Definición de recursos y herramientas

Se determinan los instrumentos y medios que se emplearán para registrar y analizar la información obtenida.

Una planeación adecuada de la elicitación contribuye a mejorar la calidad de los requisitos identificados, ya que permite abordar de forma sistemática las necesidades de los usuarios y del negocio. De esta manera, se reducen los riesgos de interpretaciones incorrectas, cambios frecuentes o inconsistencias en los requisitos durante las etapas posteriores del desarrollo de software.

5.2. Fuentes de información

Las fuentes de información corresponden a los medios o actores de los cuales se obtiene el conocimiento necesario para identificar y comprender los requisitos de un sistema. Estas fuentes permiten conocer las necesidades reales del negocio, los procesos existentes, las expectativas de los usuarios y las restricciones que pueden afectar el desarrollo del software.

Durante la elicitación de requisitos, el analista de sistemas debe identificar diferentes fuentes de información que aporten perspectivas diversas sobre el sistema a desarrollar. Utilizar múltiples fuentes contribuye a obtener una visión más completa del problema, reducir ambigüedades y validar la información recolectada.

Entre las principales fuentes de información utilizadas en la ingeniería de requisitos se encuentran:

Tabla 9. Fuentes de información para la elicitación de requisitos

Fuente de información	Descripción	Ejemplo
Usuarios finales.	Personas que utilizarán directamente el sistema y conocen las tareas que deben realizarse en el proceso operativo.	Funcionarios que registran información en un sistema de gestión académica.
Clientes o patrocinadores.	Personas o áreas responsables de solicitar el desarrollo del sistema y definir los objetivos del proyecto.	Gerencia de una empresa que solicita un sistema de control de inventarios.

Fuente de información	Descripción	Ejemplo
Expertos del dominio.	Profesionales con amplio conocimiento sobre el área de negocio donde se implementará el sistema.	Un contador que orienta los requisitos de un sistema contable.
Documentación existente.	Documentos institucionales o técnicos que describen procesos, normas o procedimientos actuales.	Manuales de operación, reglamentos, diagramas de procesos.
Sistemas existentes.	Sistemas actuales o anteriores que permiten identificar funcionalidades, procesos y limitaciones.	Un sistema antiguo de registro de clientes que se desea actualizar.
Observación del entorno de trabajo.	Análisis directo de cómo los usuarios realizan sus actividades en el contexto real.	Observar cómo un operador registra pedidos en una empresa.
Normativas y estándares.	Leyes, regulaciones o estándares técnicos que deben cumplirse en el sistema.	Normas de seguridad de la información o regulaciones de protección de datos.

El análisis de diferentes fuentes de información permite obtener una visión más completa de las necesidades del sistema. Al contrastar la información proveniente de usuarios, documentos y procesos existentes, el equipo de desarrollo puede identificar requisitos más claros, reducir ambigüedades y asegurar que el software responda de manera efectiva a las necesidades del contexto organizacional.

5.3. Técnicas de elicitación

Las técnicas de elicitación corresponden a los métodos utilizados para obtener, descubrir y comprender los requisitos del sistema a partir de las diferentes fuentes de información. Estas técnicas permiten al equipo de desarrollo interactuar con usuarios, clientes y expertos del dominio con el fin de identificar necesidades, problemas, expectativas y restricciones que deben ser consideradas durante el desarrollo del software.

La selección de la técnica adecuada depende del contexto del proyecto, la disponibilidad de los participantes, la complejidad del sistema y el tipo de información que se requiere obtener. En muchos proyectos es común combinar varias técnicas para lograr una comprensión más completa de los requisitos.

A continuación, se presentan algunas de las técnicas de elicitación más utilizadas en proyectos de desarrollo de software:

Tabla 10. Técnicas de elicitación de requisitos

Técnica	Descripción	Ejemplo de aplicación
Entrevistas.	Conversaciones estructuradas o semiestructuradas con usuarios, clientes o expertos del dominio para obtener información detallada sobre procesos, necesidades y expectativas del sistema.	Entrevistar a un jefe de área para conocer cómo se gestionan actualmente los pedidos en una empresa.
Cuestionarios o encuestas.	Instrumentos con preguntas previamente diseñadas que permiten recopilar información de un grupo amplio de personas de manera rápida.	Enviar un cuestionario a los usuarios de una plataforma para identificar dificultades en su uso.
Observación directa.	Consiste en analizar cómo los usuarios realizan sus actividades en el entorno real de trabajo para comprender los procesos y detectar necesidades.	Observar cómo un empleado registra información en un sistema de atención al cliente.
Talleres o reuniones de trabajo.	Espacios colaborativos en los que participan diferentes actores para	Reunión entre usuarios, analistas y desarrolladores

Técnica	Descripción	Ejemplo de aplicación
	discutir necesidades, procesos y funcionalidades del sistema.	para definir los requisitos de un sistema de inventario.
Análisis de documentos.	Revisión de documentos existentes que describen procesos, normas o procedimientos relacionados con el sistema.	Analizar manuales de procedimientos o reglamentos institucionales.
Prototipado.	Creación de representaciones preliminares del sistema que permiten a los usuarios visualizar funcionalidades y proponer mejoras.	Presentar un prototipo de interfaz para validar cómo se registrará la información en el sistema.

El uso adecuado de estas técnicas facilita la identificación de requisitos más claros y precisos, lo que contribuye a reducir errores durante el desarrollo del software. Asimismo, promueve la participación activa de los diferentes actores involucrados en el proyecto y mejora la comprensión de las necesidades del sistema que se desea construir.

5.4. Instrumentos de recolección de información

Los instrumentos de recolección de información corresponden a los recursos utilizados para registrar, organizar y documentar los datos obtenidos durante la aplicación de las técnicas de elicitación. Estos instrumentos permiten sistematizar la información recopilada, facilitar su análisis y asegurar que los requisitos identificados queden documentados de manera clara y estructurada.

El uso de instrumentos adecuados contribuye a mantener un registro confiable de las necesidades expresadas por los usuarios y demás participantes del proyecto. Además, permiten dar seguimiento a las decisiones tomadas durante el proceso de levantamiento de requisitos y sirven como evidencia del proceso de análisis realizado por el equipo de desarrollo.

A continuación, se presentan algunos de los instrumentos más utilizados en la recolección de información para la ingeniería de requisitos:

Tabla 11. Instrumentos de recolección de información en la elicitación de requisitos

Instrumento	Descripción	Ejemplo de uso
Guion de entrevista.	Documento que contiene las preguntas previamente estructuradas que guían el desarrollo de una entrevista con usuarios o expertos. Permite asegurar que se aborden todos	Lista de preguntas preparada para entrevistar al responsable de un proceso administrativo dentro de una organización.

Instrumento	Descripción	Ejemplo de uso
	los temas relevantes para el proyecto.	
Cuestionario.	Conjunto organizado de preguntas que se aplican a un grupo de personas con el fin de recopilar información de manera sistemática. Puede incluir preguntas abiertas o cerradas.	Formulario aplicado a los usuarios de un sistema para identificar dificultades en el uso de una plataforma.
Lista de chequeo.	Herramienta que permite verificar si determinados aspectos, requisitos o condiciones se cumplen dentro de un proceso o sistema.	Lista utilizada para confirmar que un sistema cumple con determinados requisitos funcionales o de seguridad.
Acta de reunión.	Documento que registra la información discutida durante una reunión, incluyendo acuerdos, decisiones y tareas asignadas a los participantes.	Registro de una reunión entre el cliente y el equipo de desarrollo para definir funcionalidades del sistema.

La utilización de estos instrumentos facilita la organización de la información obtenida durante el proceso de elicitación, permite mantener una trazabilidad de los requisitos identificados y contribuye a mejorar la comunicación entre los diferentes actores involucrados en el desarrollo del software.

5.5. Principios y buenas prácticas

La elicitación de requisitos es una de las actividades más críticas dentro del desarrollo de software, ya que de ella depende que el sistema responda realmente a las necesidades de los usuarios. Diversos estudios en ingeniería de software han evidenciado que una gran parte de los errores en los sistemas se originan por requisitos incompletos, ambiguos o mal interpretados durante las primeras etapas del proyecto.

Históricamente, en los primeros proyectos de desarrollo de software, especialmente entre las décadas de 1960 y 1970, era común que los equipos comenzaran a programar sin realizar una adecuada identificación de requisitos. Esta situación provocaba retrasos, sobrecostos y sistemas que no cumplían con las expectativas del cliente. A partir de estas experiencias surgieron metodologías y buenas prácticas orientadas a mejorar la captura y gestión de los requisitos.

En la actualidad, la ingeniería de requisitos establece un conjunto de principios que orientan la forma en que debe realizarse la elicitación para obtener información clara, completa y verificable.

Entre los principales principios y buenas prácticas se destacan los siguientes:

a) Comprender el contexto del sistema

Antes de recopilar requisitos, es importante conocer el entorno organizacional, los procesos existentes y las necesidades reales del negocio.

b) Involucrar a los actores adecuados

La participación de usuarios, expertos del dominio, responsables del negocio y equipo técnico permite obtener diferentes perspectivas sobre el sistema que se desea desarrollar.

c) Escuchar activamente a los usuarios

Durante entrevistas o reuniones, es fundamental prestar atención a las necesidades expresadas por los usuarios y evitar suposiciones sobre el funcionamiento del sistema.

d) Evitar ambigüedades en los requisitos

Los requisitos deben redactarse de manera clara, precisa y comprensible para todos los participantes del proyecto.

e) Validar continuamente la información obtenida

La información recopilada debe revisarse con los usuarios para confirmar que refleja correctamente sus necesidades y expectativas.

f) Documentar adecuadamente los requisitos

Mantener registros claros facilita la comunicación entre los miembros del equipo y permite realizar seguimiento durante el desarrollo del sistema.

g) Gestionar los cambios de manera controlada

Los requisitos pueden modificarse durante el proyecto; por ello, es necesario contar con mecanismos para registrar y evaluar estos cambios.

Un aspecto relevante que suele mencionarse en la ingeniería de software es que corregir un error en los requisitos puede ser hasta cien veces más costoso cuando el sistema ya está en producción. Por esta razón, dedicar tiempo y esfuerzo a una adecuada elicitación y validación de requisitos resulta fundamental para el éxito de cualquier proyecto de software.

6. Roles en la ingeniería de requisitos

La ingeniería de requisitos es un proceso colaborativo en el que participan diferentes personas con responsabilidades específicas dentro del proyecto de software. Cada uno de estos participantes aporta información, experiencia o capacidades técnicas que permiten identificar, analizar, documentar y validar los requisitos del sistema.

Los roles en la ingeniería de requisitos representan a los diferentes actores que intervienen durante la definición y gestión de las necesidades del sistema. Su participación es fundamental para garantizar que los requisitos reflejen correctamente los objetivos del negocio, las expectativas de los usuarios y las posibilidades técnicas del equipo de desarrollo.

En un proyecto de software, estos roles pueden ser desempeñados por una o varias personas, dependiendo del tamaño del proyecto, la organización del equipo y la metodología de desarrollo utilizada. En algunos casos, una misma persona puede asumir más de un rol, especialmente en proyectos pequeños o equipos reducidos.

La correcta identificación de los roles permite mejorar la comunicación entre los participantes del proyecto, facilitar la toma de decisiones y asegurar que cada requisito tenga un responsable claro para su definición, validación y seguimiento.

Entre los principales roles que intervienen en la ingeniería de requisitos se encuentran los usuarios y actores del sistema, las partes interesadas o stakeholders, el cliente líder o dueño del producto y el equipo de desarrollo junto con el analista de requisitos. Cada uno de estos participantes cumple funciones específicas que contribuyen al éxito del proceso de definición y gestión de requisitos.

6.1. Usuarios y actores

En el contexto de la ingeniería de requisitos, los usuarios y los actores representan a las personas o entidades que interactúan con el sistema de software o que se ven afectadas por su funcionamiento. Identificar correctamente a estos participantes es fundamental para comprender las necesidades reales del sistema y garantizar que el software responda a los objetivos del proyecto.

Los usuarios son las personas que utilizan el sistema de manera directa para realizar tareas específicas dentro de su entorno laboral o personal. Ellos interactúan con la interfaz del sistema, introducen información, consultan datos y ejecutan diferentes funciones que permiten cumplir los procesos definidos por la organización. Debido a su experiencia práctica en el uso del sistema, los usuarios constituyen una fuente primaria de información durante la recopilación de requisitos.

Por ejemplo, en un sistema de gestión académica, los usuarios pueden ser los aprendices que consultan sus calificaciones, los instructores que registran notas o el personal administrativo que gestiona los registros académicos.

Por su parte, los actores representan cualquier entidad que interactúa con el sistema para lograr un objetivo determinado. Un actor no necesariamente es una persona; también puede ser otro sistema informático, un dispositivo tecnológico o un servicio externo que intercambia información con el sistema en desarrollo.

Entre los tipos de actores que pueden identificarse en un proyecto de software se encuentran:

- Actores humanos, como usuarios finales, administradores del sistema, operadores o supervisores.

- Sistemas externos, como plataformas de pago, servicios de autenticación o sistemas de información de otras organizaciones.
- Dispositivos o equipos, como sensores, terminales móviles o dispositivos de registro de datos.

La identificación de usuarios y actores es especialmente importante en la elaboración de diagramas de casos de uso, ya que estos permiten representar gráficamente las interacciones entre el sistema y quienes participan en su utilización. Esta información facilita la comprensión del funcionamiento del sistema y ayuda a definir con mayor claridad las funcionalidades que debe ofrecer.

Reconocer adecuadamente a los usuarios y actores contribuye a diseñar sistemas más funcionales, comprensibles y alineados con las necesidades reales de quienes interactúan con la solución tecnológica.

6.2. Stakeholders o partes interesadas

En el desarrollo de software, los stakeholders o partes interesadas corresponden a todas las personas, grupos u organizaciones que tienen algún interés en el sistema o que pueden verse afectados por su desarrollo, implementación o funcionamiento. Su participación resulta fundamental durante la ingeniería de requisitos, ya que proporcionan información clave sobre las necesidades del negocio, las restricciones del proyecto y las expectativas del sistema.

Las partes interesadas no siempre utilizan directamente el sistema, pero influyen en las decisiones relacionadas con su diseño, alcance y funcionamiento. Por esta razón, identificar a los stakeholders desde las etapas iniciales del proyecto permite

comprender mejor el contexto organizacional y reducir riesgos asociados a requisitos incompletos o mal interpretados.

Entre los principales tipos de stakeholders que pueden participar en un proyecto de software se encuentran:

- Usuarios finales, quienes interactúan directamente con el sistema en sus actividades cotidianas.
- Clientes o patrocinadores del proyecto, responsables de financiar o solicitar el desarrollo del sistema.
- Directivos o gerentes, quienes toman decisiones estratégicas relacionadas con el proyecto.
- Equipo técnico, incluyendo desarrolladores, arquitectos de software y especialistas en infraestructura.
- Áreas de soporte o mantenimiento, encargadas de operar y mantener el sistema una vez implementado.
- Entidades regulatorias o legales, que establecen normas o requisitos de cumplimiento.

La correcta identificación y gestión de los stakeholders permite establecer canales de comunicación efectivos y asegurar que las necesidades del negocio se reflejen adecuadamente en los requisitos del sistema. Además, facilita la validación de los resultados obtenidos durante el desarrollo del software.

En muchos proyectos de ingeniería de requisitos se recomienda elaborar una matriz de stakeholders, herramienta que permite identificar a cada parte interesada,

su nivel de influencia en el proyecto, sus expectativas y el tipo de participación que tendrá durante el proceso de desarrollo del sistema.

A continuación, se presenta la relación entre los actores involucrados, su nivel de influencia e interés y la forma en que participan en el proyecto:

Tabla 12. Ejemplo de matriz de stakeholders en un proyecto de software

Stakeholder	Rol o relación con el sistema	Nivel de influencia	Interés en el proyecto	Forma de participación
Cliente o patrocinador.	Solicita y financia el desarrollo del sistema.	Alto	Alto	Define objetivos, aprueba requisitos y valida entregables.
Usuarios finales.	Utilizan el sistema en sus actividades diarias.	Medio	Alto	Proporcionan necesidades, participan en entrevistas y pruebas.
Gerencia o directivos.	Toman decisiones estratégicas sobre el proyecto.	Alto	Medio	Revisan avances y aprueban decisiones clave.

Stakeholder	Rol o relación con el sistema	Nivel de influencia	Interés en el proyecto	Forma de participación
Equipo de desarrollo.	Diseña, construye y prueba el sistema.	Medio	Alto	Analiza requisitos, implementa funcionalidades y realiza pruebas.
Área de soporte técnico.	Mantiene el sistema después de su implementación.	Bajo	Medio	Aporta información sobre mantenimiento y operación del sistema.

La elaboración de una matriz de stakeholders facilita la identificación de las personas clave que deben participar durante la ingeniería de requisitos. Además, permite definir estrategias de comunicación y participación adecuadas para cada grupo, lo que contribuye a mejorar la comprensión de las necesidades del sistema y a reducir riesgos durante el desarrollo del software.

6.3. Cliente líder y dueño del producto

Dentro de la ingeniería de requisitos, el cliente líder y el dueño del producto cumplen un papel fundamental en la definición de las necesidades del sistema y en la toma de decisiones relacionadas con el desarrollo del software. Estas figuras

representan los intereses del negocio y aseguran que el sistema que se construye responda a los objetivos organizacionales y a las necesidades reales de los usuarios.

El cliente líder corresponde a la persona o representante de la organización que solicita el desarrollo del sistema y actúa como principal punto de contacto entre el equipo de desarrollo y la entidad que requiere la solución tecnológica. Su función consiste en proporcionar información sobre el contexto del negocio, definir prioridades, validar los requisitos identificados y aprobar los entregables generados durante el proyecto.

Entre las principales responsabilidades del cliente líder se encuentran:

- Definir los objetivos generales del sistema.
- Proporcionar información sobre los procesos del negocio.
- Validar que los requisitos identificados respondan a las necesidades de la organización.
- Aprobar los entregables generados durante el desarrollo del software.
- Facilitar la comunicación entre los diferentes actores del proyecto.

Por su parte, el dueño del producto (product owner) es un rol ampliamente utilizado en metodologías ágiles de desarrollo de software. Esta persona es responsable de representar los intereses de los usuarios y del negocio dentro del equipo de desarrollo, asegurando que el producto evolucione de acuerdo con las prioridades definidas.

El dueño del producto gestiona y prioriza las funcionalidades que debe tener el sistema, organiza los requisitos en un listado estructurado de trabajo y toma decisiones

sobre qué características se desarrollarán primero. De esta manera, orienta el trabajo del equipo hacia la generación de valor para el usuario y para la organización.

Entre las funciones principales del dueño del producto se destacan:

- Definir y priorizar las funcionalidades del sistema.
- Representar las necesidades de los usuarios y del negocio.
- Mantener actualizado el listado de requisitos o backlog del producto.
- Validar que las funcionalidades desarrolladas cumplan con los requisitos establecidos.
- Colaborar continuamente con el equipo de desarrollo para aclarar dudas sobre los requisitos.

La participación activa del cliente líder y del dueño del producto permite mantener una comunicación constante entre el equipo técnico y la organización que solicita el sistema. Esta interacción facilita la toma de decisiones, mejora la comprensión de los requisitos y contribuye a que el software desarrollado cumpla con las expectativas del negocio y de los usuarios finales.

6.4. Equipo de desarrollo y analista

El equipo de desarrollo y el analista de requisitos desempeñan un papel esencial en la ingeniería de requisitos, ya que son responsables de interpretar, analizar y transformar las necesidades del negocio en soluciones tecnológicas que puedan implementarse mediante software. Su trabajo permite convertir las expectativas de los usuarios y las organizaciones en especificaciones técnicas claras y viables.

El equipo de desarrollo está conformado por los profesionales encargados de diseñar, construir, probar e implementar el sistema. Este equipo puede incluir desarrolladores de software, arquitectos de sistemas, diseñadores de interfaces, especialistas en bases de datos y profesionales de pruebas o calidad. Cada uno de estos roles contribuye desde su especialidad al desarrollo del producto tecnológico.

Entre las principales responsabilidades del equipo de desarrollo se encuentran:

- Analizar los requisitos definidos durante la ingeniería de requisitos.
- Diseñar la arquitectura y la estructura técnica del sistema.
- Implementar las funcionalidades mediante programación.
- Realizar pruebas para verificar el correcto funcionamiento del software.
- Participar en la mejora continua del sistema durante su mantenimiento.

Por su parte, el analista de requisitos cumple una función de enlace entre el negocio y el equipo técnico. Este profesional se encarga de comprender las necesidades de los usuarios, documentarlas de manera estructurada y asegurar que dichas necesidades sean interpretadas correctamente por el equipo de desarrollo.

El analista también facilita la comunicación entre las diferentes partes interesadas, organiza la información obtenida durante la elicitación de requisitos y valida que los requisitos definidos sean completos, claros y consistentes.

Entre las funciones principales del analista de requisitos se destacan:

- Identificar y comprender las necesidades de los usuarios y del negocio.
- Documentar los requisitos de manera clara y estructurada.
- Apoyar el proceso de elicitación mediante entrevistas, reuniones y talleres.
- Verificar que los requisitos sean consistentes, completos y comprensibles.

- Servir como puente de comunicación entre los stakeholders y el equipo de desarrollo.

La colaboración permanente entre el analista de requisitos y el equipo de desarrollo permite asegurar que las funcionalidades implementadas respondan correctamente a las necesidades del sistema. Además, facilita la resolución de dudas durante el proceso de desarrollo y contribuye a mejorar la calidad del software construido.

7. Herramientas de modelado de requisitos

Las herramientas de modelado de requisitos son recursos que permiten representar de manera gráfica y estructurada la información obtenida durante la ingeniería de requisitos. Su utilización facilita la comprensión de las necesidades del sistema, mejora la comunicación entre los miembros del equipo y permite documentar de forma clara el comportamiento esperado del software.

En el desarrollo de proyectos de software, el modelado de requisitos se utiliza para describir cómo interactúan los usuarios con el sistema, qué procesos se ejecutan, qué información se maneja y cómo se relacionan los diferentes componentes del sistema. Estas representaciones permiten transformar descripciones textuales complejas en esquemas visuales que resultan más fáciles de interpretar.

Las herramientas de modelado ayudan a organizar la información del proyecto, identificar relaciones entre los diferentes elementos del sistema y detectar posibles inconsistencias en los requisitos definidos. Además, permiten que los diferentes actores del proyecto, como analistas, desarrolladores, diseñadores y usuarios, compartan una visión común sobre el funcionamiento del sistema.

Entre los beneficios más importantes del uso de herramientas de modelado en la ingeniería de requisitos se destacan:

- Facilitan la comprensión de los requisitos mediante representaciones visuales.
- Permiten estructurar y organizar la información del sistema.
- Mejoran la comunicación entre los miembros del equipo de proyecto.
- Apoyan la validación de requisitos con usuarios y stakeholders.

- Contribuyen a identificar errores o inconsistencias en etapas tempranas del desarrollo.

El uso de herramientas de modelado resulta especialmente útil en proyectos de software complejos, donde intervienen múltiples actores, procesos y componentes tecnológicos. En estos casos, los modelos permiten representar de forma clara las interacciones del sistema y sirven como base para las etapas posteriores de diseño e implementación.

En los siguientes apartados se presentan los principales tipos de herramientas de modelado utilizadas en la ingeniería de requisitos, así como las características que las hacen útiles dentro del proceso de desarrollo de software.

7.1. Tipos de herramientas de modelado

Las herramientas de modelado de requisitos son aplicaciones o recursos que permiten representar de forma visual y estructurada los diferentes elementos de un sistema de software. Estas herramientas facilitan la comprensión del sistema, ya que permiten organizar la información, identificar relaciones entre componentes y comunicar de manera clara los requisitos definidos durante el proceso de desarrollo.

El uso de herramientas de modelado permite representar gráficamente procesos, estructuras de datos, interacciones entre actores y comportamiento del sistema. Gracias a estas representaciones, los equipos de desarrollo pueden analizar con mayor claridad el funcionamiento del sistema antes de iniciar la fase de implementación.

Existen diferentes tipos de herramientas de modelado, las cuales se utilizan dependiendo del tipo de información que se desea representar y del enfoque metodológico del proyecto.

Entre los principales tipos de herramientas de modelado utilizadas en ingeniería de software se encuentran:

a) Herramientas de modelado UML

Permiten representar diferentes aspectos del sistema mediante diagramas estandarizados del Lenguaje Unificado de Modelado (UML). Estas herramientas facilitan la creación de diagramas de casos de uso, diagramas de clases, diagramas de secuencia, diagramas de actividades y diagramas de componentes.

b) Herramientas de modelado de procesos

Se utilizan para representar el flujo de actividades dentro de un sistema o proceso de negocio. Estas herramientas permiten visualizar cómo se desarrollan las tareas, qué actores participan y cómo se relacionan las diferentes actividades del proceso.

c) Herramientas de modelado de datos

Permiten representar la estructura de los datos que utilizará el sistema. Mediante estas herramientas se elaboran diagramas entidad-relación, modelos de bases de datos y estructuras de almacenamiento de información.

d) Herramientas de prototipado

Facilitan la creación de representaciones visuales de las interfaces del sistema antes de su desarrollo definitivo. A través de prototipos se pueden evaluar aspectos de usabilidad, navegación y diseño de la interfaz.

e) Herramientas de modelado colaborativo

Permiten que varios miembros del equipo trabajen simultáneamente en la construcción de modelos del sistema. Estas herramientas facilitan la comunicación entre los participantes del proyecto y el seguimiento de cambios realizados en los modelos.

A continuación, se presenta una tabla con algunos ejemplos de herramientas utilizadas para el modelado de sistemas en proyectos de software:

Tabla 13. Ejemplos de herramientas de modelado de requisitos

Tipo de herramienta	Ejemplos	Uso principal
Modelado UML.	StarUML, Visual Paradigm y Enterprise Architect.	Elaboración de diagramas del sistema.
Modelado de procesos.	Bizagi Modeler y Lucidchart.	Representación de procesos y flujos de trabajo.
Modelado de datos.	MySQL Workbench y ER/Studio.	Diseño de bases de datos.

Tipo de herramienta	Ejemplos	Uso principal
Prototipado.	Figma, Balsamiq y Adobe XD.	Diseño de interfaces y prototipos.
Modelado colaborativo.	Miro, Draw.io y Lucidchart.	Trabajo colaborativo en diagramas.

La utilización de herramientas de modelado contribuye a mejorar la comprensión del sistema y facilita la comunicación entre los miembros del equipo de desarrollo y las partes interesadas. Además, permite documentar de manera clara la estructura y el comportamiento del software antes de iniciar su construcción.

7.2. Características de las herramientas de modelado

Las herramientas de modelado de requisitos presentan un conjunto de características que facilitan la representación, análisis y documentación de los diferentes elementos de un sistema de software. Estas herramientas permiten estructurar la información del proyecto, visualizar relaciones entre componentes y apoyar la toma de decisiones durante las etapas iniciales del desarrollo.

Una de sus principales características es la representación gráfica, ya que permiten expresar ideas, procesos y estructuras del sistema mediante diagramas y esquemas visuales. Este tipo de representación facilita la comprensión del sistema tanto para el equipo técnico como para los usuarios o partes interesadas que participan en el proyecto.

Otra característica importante es la estandarización de modelos, lo que significa que muchas herramientas utilizan notaciones reconocidas internacionalmente, como el Lenguaje Unificado de Modelado (UML) o modelos de procesos. Esto permite que los diagramas elaborados puedan ser comprendidos por diferentes profesionales del área de desarrollo de software.

Las herramientas de modelado también ofrecen facilidades de documentación, ya que permiten almacenar información relacionada con los elementos del sistema, como descripciones de requisitos, relaciones entre componentes, reglas de negocio y especificaciones técnicas. De esta manera, se genera una base de conocimiento que puede ser utilizada durante todo el ciclo de vida del software.

Asimismo, muchas herramientas cuentan con funcionalidades de trabajo colaborativo, lo que permite que varios miembros del equipo participen en la construcción y revisión de los modelos del sistema. Esta característica favorece la comunicación entre analistas, desarrolladores, diseñadores y otros actores involucrados en el proyecto.

Otra característica relevante es la capacidad de actualización y trazabilidad, ya que las herramientas permiten modificar los modelos cuando cambian los requisitos del sistema y mantener un registro de las versiones o cambios realizados. Esto facilita el control del proyecto y la gestión de los requisitos a lo largo del desarrollo.

A continuación, se presentan algunas características comunes de las herramientas de modelado utilizadas en proyectos de software:

Tabla 14. Características de las herramientas de modelado de requisitos

Característica	Descripción
Representación visual.	Permiten expresar el funcionamiento del sistema mediante diagramas y modelos gráficos.
Uso de estándares.	Utilizan notaciones reconocidas como UML u otros modelos de representación de sistemas.
Documentación estructurada.	Facilitan el registro y organización de la información relacionada con los requisitos del sistema.
Trabajo colaborativo.	Permiten que varios miembros del equipo participen en la creación y revisión de los modelos.
Trazabilidad de cambios.	Registran modificaciones realizadas en los modelos y permiten gestionar versiones del proyecto.
Integración con otras herramientas.	Algunas herramientas se integran con sistemas de desarrollo, gestión de proyectos o repositorios de código.

Estas características convierten a las herramientas de modelado en un recurso fundamental dentro de la ingeniería de requisitos, ya que facilitan la comprensión del sistema, mejoran la comunicación entre los participantes del proyecto y permiten documentar de manera clara la estructura y el comportamiento del software.

8. Herramientas para la captura de requisitos

En el desarrollo de software, la captura de requisitos constituye una actividad fundamental dentro de la ingeniería de requisitos, ya que permite identificar, comprender y documentar las necesidades que deben ser atendidas por el sistema. Para facilitar este proceso, se utilizan diversas herramientas que ayudan a representar la información de forma clara, organizada y comprensible para todos los participantes del proyecto.

Las herramientas para la captura de requisitos permiten describir cómo interactúan los usuarios con el sistema, cuáles son las funcionalidades esperadas y de qué manera se deben atender las necesidades del negocio. Estas herramientas favorecen la comunicación entre analistas, desarrolladores, usuarios y demás partes interesadas, lo que contribuye a reducir ambigüedades y mejorar la calidad de los requisitos definidos.

Desde los inicios de la ingeniería de software, diferentes métodos han sido utilizados para representar los requisitos de un sistema. En los primeros proyectos informáticos, la documentación se realizaba principalmente mediante descripciones textuales extensas. Con el paso del tiempo, surgieron enfoques más estructurados que incorporaron diagramas, modelos visuales y representaciones narrativas que facilitan la comprensión de las funcionalidades del sistema.

Durante la década de 1990, el uso de diagramas de modelado se consolidó con la aparición de metodologías orientadas a objetos y del Lenguaje Unificado de Modelado (UML), que permitió representar gráficamente las interacciones entre los usuarios y el sistema. Posteriormente, con la adopción de metodologías ágiles en el desarrollo de

software, se popularizó el uso de historias de usuario como una forma sencilla y flexible de describir las necesidades del usuario final.

En la actualidad, las organizaciones combinan diferentes herramientas de captura de requisitos para obtener una visión más completa del sistema que se desea construir. Algunas herramientas permiten representar las interacciones del sistema, otras describen las funcionalidades desde la perspectiva del usuario y otras ayudan a visualizar la experiencia del usuario a través de secuencias gráficas o narrativas.

Entre las herramientas más utilizadas para la captura de requisitos se encuentran los diagramas de casos de uso, las historias de usuario y los storyboard, las cuales permiten representar la funcionalidad del sistema desde diferentes perspectivas y facilitan la comunicación entre los diferentes actores del proyecto.

Estas herramientas se analizan en los siguientes apartados, donde se describe su propósito, características y la forma en que contribuyen a la documentación y comprensión de los requisitos del sistema.

8.1. Diagramas de casos de uso

Los diagramas de casos de uso constituyen una de las herramientas más utilizadas para representar los requisitos funcionales de un sistema. Su propósito principal consiste en describir las interacciones que se establecen entre los usuarios y el sistema, permitiendo comprender qué funcionalidades ofrece el software y cómo se utilizan en diferentes situaciones.

Este tipo de diagramas forma parte del Lenguaje Unificado de Modelado (UML) y se emplea ampliamente en la ingeniería de software para representar de manera gráfica las acciones que el sistema debe realizar desde la perspectiva del usuario. En lugar de centrarse en aspectos técnicos o en la estructura interna del sistema, los diagramas de casos de uso se enfocan en el comportamiento observable del sistema y en las actividades que los usuarios pueden ejecutar.

Los diagramas de casos de uso permiten identificar claramente quién interactúa con el sistema, qué funciones están disponibles y cómo se relacionan las diferentes funcionalidades. De esta manera, facilitan la comunicación entre analistas, desarrolladores, usuarios y demás partes interesadas, contribuyendo a que todos los participantes del proyecto compartan una visión común sobre el funcionamiento del sistema.

Un diagrama de casos de uso está compuesto por varios elementos que permiten representar las interacciones entre los actores y el sistema:

- **Actor:** corresponde a la persona, sistema externo o entidad que interactúa con el sistema. Los actores representan los diferentes tipos de usuarios que utilizan las funcionalidades del software.
- **Caso de uso:** representa una funcionalidad o servicio que el sistema ofrece a los usuarios. Cada caso de uso describe una acción específica que el sistema debe realizar.
- **Sistema:** se representa mediante un rectángulo que delimita el alcance del sistema. Dentro de este límite se ubican los casos de uso que forman parte del sistema.

- **Relaciones:** Los actores se conectan con los casos de uso mediante líneas que representan las interacciones. En algunos casos pueden existir relaciones adicionales entre casos de uso, como inclusión o extensión de funcionalidades.

La construcción de un diagrama de casos de uso suele desarrollarse mediante una serie de pasos que permiten identificar las funcionalidades principales del sistema y las interacciones con los usuarios.

a) Identificar los actores del sistema

En primer lugar, se determinan las personas o sistemas externos que interactúan con el software. Los actores pueden ser usuarios finales, administradores, sistemas externos o dispositivos.

b) Identificar los casos de uso

Posteriormente, se establecen las funcionalidades que el sistema debe ofrecer. Cada caso de uso representa una acción que el sistema ejecuta para satisfacer una necesidad del usuario.

c) Definir el límite del sistema

Se delimita el sistema mediante un rectángulo dentro del cual se ubican los casos de uso. Este límite permite diferenciar lo que pertenece al sistema y lo que corresponde a los actores externos.

d) Establecer las relaciones entre actores y casos de uso

Se conectan los actores con los casos de uso que utilizan mediante líneas que representan la interacción.

e) Revisar y validar el diagrama

Finalmente, se revisa el diagrama con los stakeholders del proyecto para confirmar que las funcionalidades representadas corresponden a las necesidades del sistema.

Ejemplo de diagrama de casos de uso:

En un sistema de comercio electrónico pueden identificarse diferentes actores y funcionalidades.

- **Actor:** cliente.
- **Casos de uso:**
 - Registrarse en el sistema.
 - Iniciar sesión.
 - Consultar productos.
 - Agregar productos al carrito.
 - Realizar compra.
 - Consultar historial de pedidos.
- **Actor:** administrador.
- **Casos de uso:**
 - Gestionar productos.
 - Actualizar inventario.
 - Consultar reportes de ventas.
 - Administrar usuarios.

En este ejemplo, el cliente interactúa con el sistema para realizar compras, mientras que el administrador gestiona la información del sistema.

Actualmente existen diversas herramientas que permiten elaborar diagramas de casos de uso de manera gráfica y estructurada.

A continuación, se presentan algunas opciones disponibles y sus principales características para apoyar la representación gráfica de los requisitos del sistema:

Tabla 15. Herramientas para crear diagramas de casos de uso

Herramienta	Características
Lucidchart	Plataforma en línea que permite crear diagramas UML de manera colaborativa.
Draw.io (diagrams.net)	Herramienta gratuita para crear diagramas técnicos y modelos UML.
StarUML	Software especializado en modelado UML utilizado en proyectos de desarrollo de software.
Visual Paradigm	Plataforma profesional para modelado de sistemas y gestión de proyectos de software.
Microsoft Visio	Herramienta de diagramación que permite crear diferentes tipos de modelos técnicos.

El uso de diagramas de casos de uso permite comprender de forma clara las funcionalidades que debe ofrecer el sistema y las interacciones con los usuarios. Además, facilita la identificación de requisitos funcionales, mejora la comunicación entre los miembros del equipo y sirve como base para el diseño y desarrollo del software.

8.2. Historias de usuario

Las historias de usuario constituyen una técnica ampliamente utilizada en metodologías ágiles para describir los requisitos del sistema desde la perspectiva de quienes interactúan con él. Su propósito es expresar de manera clara y sencilla una necesidad funcional del usuario, enfocándose en el valor que el sistema debe proporcionar y evitando descripciones técnicas complejas en las etapas iniciales del desarrollo.

Este enfoque surge en el marco de las metodologías ágiles, especialmente dentro de la metodología Extreme Programming, y posteriormente se consolida como una práctica fundamental en Scrum framework, donde las historias de usuario permiten organizar y priorizar los requisitos del producto durante el proceso de desarrollo.

Una historia de usuario se redacta normalmente siguiendo una estructura simple que permite identificar tres elementos fundamentales: la persona que necesita la funcionalidad, la acción que desea realizar y el beneficio que obtiene. De forma general, se expresa mediante la siguiente estructura:

- Como [tipo de usuario] + Quiero [funcionalidad o acción] + Para [obtener un beneficio o resultado]

Este formato permite que los analistas, desarrolladores y demás participantes del proyecto comprendan fácilmente el propósito de cada funcionalidad del sistema y mantengan el enfoque en las necesidades reales de los usuarios.

Las historias de usuario suelen complementarse con criterios de aceptación, los cuales describen las condiciones que deben cumplirse para considerar que el requisito ha sido implementado correctamente. Estos criterios facilitan la validación del desarrollo y permiten verificar si la funcionalidad satisface las necesidades planteadas.

Para elaborar una historia de usuario de manera adecuada, se puede seguir el siguiente procedimiento:

a) Identificar el tipo de usuario

Se determina quién interactuará con el sistema, por ejemplo, un cliente, administrador, estudiante o visitante.

b) Definir la funcionalidad requerida

Se establece la acción que el usuario necesita realizar dentro del sistema, como consultar información, registrar datos o realizar una transacción.

c) Determinar el beneficio esperado

Se explica el propósito de la funcionalidad o el valor que aporta al usuario.

d) Redactar la historia de usuario

Se estructura la historia utilizando el formato estándar para que sea comprensible para todos los miembros del equipo.

e) Establecer los criterios de aceptación

Se describen las condiciones que permitirán verificar que la funcionalidad se implementó correctamente.

A continuación, se presentan algunos ejemplos que explican cómo se redactan las historias de usuario en diferentes contextos.

Ejemplo 1. Plataforma educativa

- Historia de usuario: como aprendiz de un curso virtual, quiero acceder a los contenidos de cada módulo desde una plataforma en línea, para estudiar los materiales del curso y avanzar en mi proceso de formación.

Criterios de aceptación:

- El aprendiz puede ingresar a la plataforma con usuario y contraseña.
- El sistema presenta los módulos organizados por unidades.
- El aprendiz puede abrir los contenidos y revisar los recursos educativos.
- El sistema registra el avance del aprendiz.

Ejemplo 2. Sistema de tienda virtual

- Historia de usuario: como cliente de una tienda en línea, quiero agregar productos a un carrito de compras, para realizar posteriormente el pago de los artículos seleccionados.

Criterios de aceptación:

- El cliente puede seleccionar un producto desde el catálogo.

- El sistema permite añadir el producto al carrito de compras.
- El carrito presenta el listado de productos seleccionados y su precio.
- El cliente puede modificar o eliminar productos antes de realizar el pago.

De esta manera, las historias de usuario permiten describir los requisitos del sistema de forma clara, centrada en el usuario y orientada al valor que cada funcionalidad aporta al producto desarrollado.

8.3. Storyboard

El storyboard es una herramienta visual utilizada en la captura y análisis de requisitos que permite representar, mediante una secuencia de escenas o ilustraciones, la forma en que un usuario interactúa con un sistema o aplicación. Su propósito es describir de manera gráfica el flujo de acciones que realiza el usuario, facilitando la comprensión del funcionamiento del sistema antes de su desarrollo.

El uso del storyboard tiene origen en la industria cinematográfica, especialmente en los procesos de planificación de animación desarrollados por la empresa Walt Disney Productions, donde se utilizaban secuencias de dibujos para planificar las escenas de una película. Posteriormente, esta técnica fue adoptada en áreas como el diseño de software, la experiencia de usuario y el desarrollo de productos digitales, debido a su capacidad para representar procesos de interacción de forma clara y comprensible.

En ingeniería de requisitos, el storyboard permite presentar cómo se desarrolla una situación de uso del sistema paso a paso. Cada escena representa una acción o

momento específico dentro del proceso, lo que facilita la identificación de funcionalidades necesarias, posibles errores o mejoras en la experiencia del usuario.

Para elaborar un storyboard en el contexto del desarrollo de software, se pueden seguir los siguientes pasos:

a) Definir el escenario de uso

Se identifica la situación o proceso que se desea representar, por ejemplo, el registro de un usuario, la compra de un producto o el acceso a información dentro de un sistema.

b) Identificar al usuario o actor principal

Se determina quién interactúa con el sistema dentro del escenario planteado.

c) Establecer la secuencia de acciones

Se describen los pasos que realiza el usuario durante su interacción con el sistema.

d) Representar visualmente cada escena

Cada paso del proceso se representa mediante imágenes, bocetos o diagramas que muestran la interacción entre el usuario y la interfaz del sistema.

e) Analizar y validar el flujo de interacción

El equipo de desarrollo revisa el storyboard para identificar mejoras, validar los requisitos y garantizar que el sistema responda adecuadamente a las necesidades del usuario.

A continuación, se presenta un ejemplo simplificado de storyboard aplicado a un sistema de inscripción a cursos en línea:

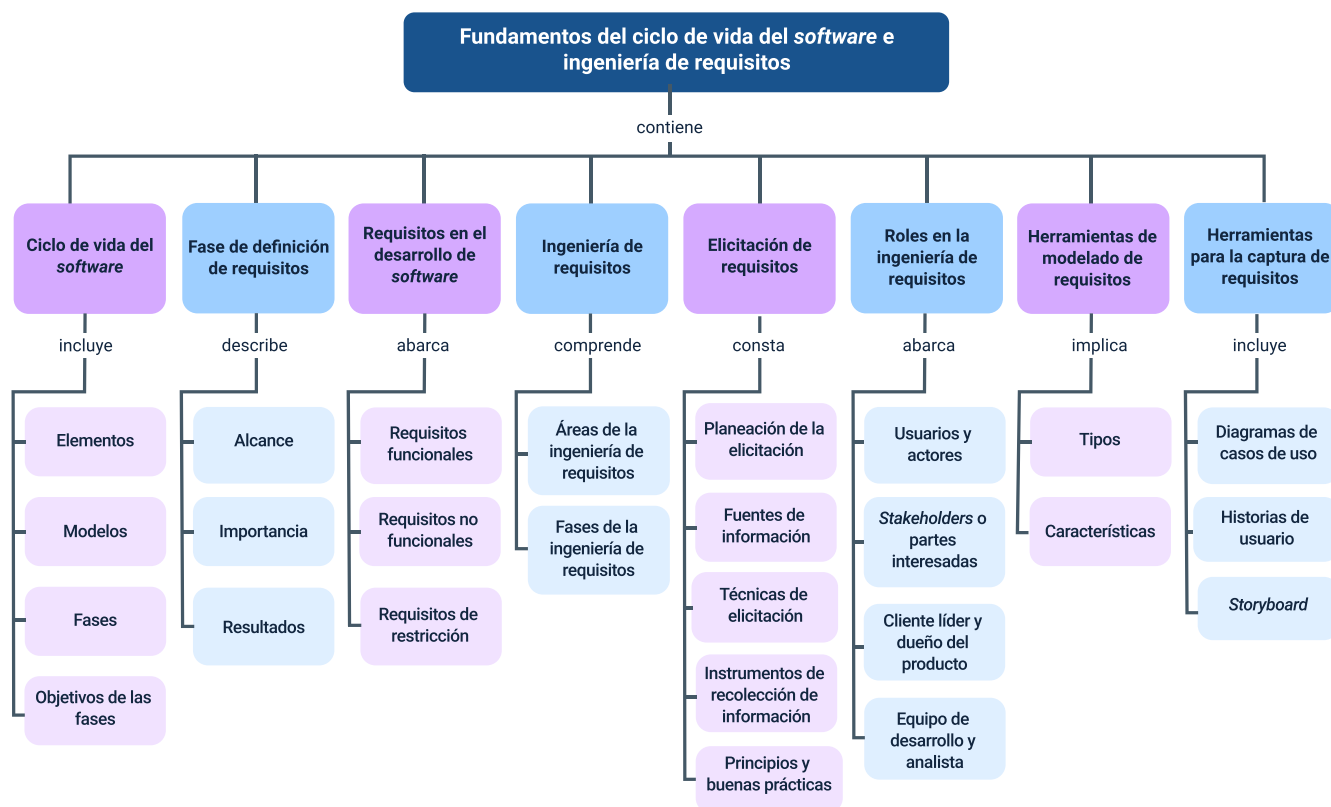
Tabla 16. Ejemplo de storyboard

Escena	Descripción de la acción
1	El usuario ingresa a la plataforma educativa desde su navegador.
2	El usuario introduce su nombre de usuario y contraseña para iniciar sesión.
3	El sistema presenta el panel principal con los cursos disponibles.
4	El usuario selecciona un curso de su interés.
5	El sistema presenta la información del curso y la opción de inscripción.
6	El usuario confirma la inscripción y el sistema registra el curso en su perfil.

Este tipo de representación permite que analistas, desarrolladores y usuarios comprendan con mayor facilidad cómo se desarrollará la interacción con el sistema, lo que contribuye a mejorar la definición de los requisitos y a reducir errores durante el proceso de desarrollo del software.

Síntesis

El componente formativo aborda los fundamentos del ciclo de vida del software y el papel de la ingeniería de requisitos en el desarrollo de sistemas informáticos. A lo largo del contenido se analizan las etapas que permiten planificar, analizar y construir soluciones tecnológicas, iniciando con la definición y gestión de requisitos, su importancia y los resultados que se obtienen durante esta fase. Asimismo, se presentan los diferentes tipos de requisitos, los roles que intervienen en el proceso y las fases de la ingeniería de requisitos. Finalmente, se exploran técnicas, fuentes de información y herramientas utilizadas para la captura, modelado y documentación de requisitos, tales como diagramas de casos de uso, historias de usuario y storyboard, que facilitan la comprensión de las necesidades del sistema y su adecuada implementación en contextos reales de desarrollo de software.



Glosario

Actor: entidad externa que interactúa con el sistema para lograr un objetivo específico dentro de un caso de uso. Puede ser una persona, otro sistema o un dispositivo.

Analista de requisitos: profesional encargado de identificar, documentar, analizar y validar las necesidades del negocio y de los usuarios para transformarlas en requisitos del sistema.

Caso de uso: descripción estructurada de las interacciones entre un actor y el sistema con el fin de lograr una funcionalidad específica.

Elicitación de requisitos: proceso mediante el cual se identifican, recopilan y comprenden las necesidades y expectativas de los usuarios y demás partes interesadas.

Historia de usuario: descripción breve de una funcionalidad del sistema desde la perspectiva del usuario, utilizada comúnmente en metodologías ágiles.

Ingeniería de requisitos: disciplina de la ingeniería de software encargada de identificar, analizar, documentar y gestionar los requisitos de un sistema.

Modelado: representación conceptual o gráfica de un sistema, sus componentes y sus relaciones para facilitar su comprensión y análisis.

Requisito: condición, capacidad o característica que debe cumplir un sistema para satisfacer una necesidad del usuario o del negocio.

Requisito de restricción: limitación técnica, normativa o tecnológica que condiciona el diseño o funcionamiento del sistema.

Requisito funcional: describe las funciones o comportamientos que el sistema debe ejecutar para responder a las necesidades del usuario.

Requisito no funcional: define las condiciones de calidad o restricciones bajo las cuales debe operar el sistema, como rendimiento, seguridad o usabilidad.

Stakeholder: persona, grupo u organización que tiene interés en el sistema o que puede verse afectado por su desarrollo o implementación.

Storyboard: representación visual que describe de manera secuencial cómo un usuario interactúa con el sistema a través de pantallas o escenarios.

UML (Lenguaje Unificado de Modelado): lenguaje estándar utilizado para representar gráficamente sistemas de software mediante diferentes tipos de diagramas.

Validación de requisitos: proceso mediante el cual se verifica que los requisitos definidos representan correctamente las necesidades de los usuarios y del negocio.

Referencias bibliográficas

IEEE Computer Society. (2014). Guía SWEBOK: Guía para el cuerpo de conocimiento de la ingeniería de software (Versión 3.0). IEEE Computer Society.

ISO/IEC. (2018). ISO/IEC/IEEE 29148:2018 Systems and software engineering - Life cycle processes - Requirements engineering. International Organization for Standardization.

Pressman, R. S., & Maxim, B. R. (2016). Ingeniería del software: Un enfoque práctico (8.ª ed.). McGraw-Hill Education.

Sommerville, I. (2011). Ingeniería del software (9.ª ed.). Pearson Educación.

Universidad Nacional de Colombia. (s. f.). Ingeniería de requisitos en el desarrollo de software.

Wiegers, K., & Beatty, J. (2014). Ingeniería de requisitos de software (3.ª ed.). Microsoft Press.

Wazlawick, R. S. (2015). Análisis y diseño de sistemas de información orientados a objetos. Elsevier.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Viviana Esperanza Herrera Quiñonez	Evaluadora instruccional	Centro de Comercio y Servicios - Regional Tolima
Andrés Felipe Velandia Espitia	Evaluador instruccional	Centro de Comercio y Servicios - Regional Tolima
Jose Yobani Penagos Mora	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Oscar Ivan Uribe Ortiz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Manuel Felipe Echavarria Orozco	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
Jorge Eduardo Rueda Peña	Evaluador de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
María Fernanda Pineda Mora	Evaluadora de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima